

AuditPlatform

Rapport d'analyse — plateforme-audit-ia-2

Généré le 26/03/2026 à 21:38

■ Informations du projet

Nom du projet	plateforme-audit-ia-2
URL du projet	yosrchiha01/plateforme-audit-ia-2
Branche analysée	main
Date de l'analyse	26/03/2026 21:26
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Désactivée

■ Scores

Qualité	76	76/100
Sécurité	0	0/100
Performance	65	65/100

■■ Vulnérabilités (87)

Complexité

Sévérité : HAUTE

Fichier : backend/app/routes/analyses.py — ligne 0

Correction : Réduire la complexité de la fonction lancer_analyse en utilisant des fonctions plus petites et plus faciles à comprendre.

Requête HTTP non fermée

Sévérité : HAUTE

Fichier : backend/app/routes/analyses.py — ligne 123

Correction : Utiliser un gestionnaire de contexte pour fermer la connexion HTTP

Requête HTTP non fermée

Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 456

Correction : Utiliser un gestionnaire de contexte pour fermer la connexion HTTP

Requêtes exécutées à l'intérieur de boucles

Sévérité : HAUTE

Fichier : frontend/app/Admin_page/page.tsx — ligne 456

Correction : Utiliser des requêtes SQL avec des paramètres pour éviter les injections SQL

SOLID-O

Sévérité : HAUTE

Fichier : frontend/app/Admin_page/page.tsx — ligne 0

Correction : La classe `AdminDashboard` est trop grande et complexe. Il est préférable de la décomposer en plusieurs classes plus petites et plus faciles à comprendre.

SOLID-O

Sévérité : HAUTE

Fichier : frontend/app/analyse/page.tsx — ligne 0

Correction : La classe `AnalysePage` est trop grande et complexe. Il est préférable de la décomposer en plusieurs classes plus petites et plus faciles à comprendre.

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/codedepots/page.tsx — ligne 34

Correction : Utiliser des jointures pour réduire le nombre de requêtes SQL

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/dashboard/page.tsx — ligne 34

Correction : Utiliser des jointures pour réduire le nombre de requêtes SQL

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/depots/page.tsx — ligne 34

Correction : Utiliser des jointures pour réduire le nombre de requêtes SQL

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/codedepots/page.tsx — ligne 120

Correction : Il est préférable de gérer les erreurs de manière plus spécifique et de ne pas utiliser de blocs catch vides.

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/dashboard/page.tsx — ligne 64

Correction : Il est préférable de gérer les erreurs de manière plus spécifique et de ne pas utiliser de blocs catch vides.

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/depots/page.tsx — ligne 40

Correction : Il est préférable de gérer les erreurs de manière plus spécifique et de ne pas utiliser de blocs catch vides.

Maintenabilité

Sévérité : HAUTE

Fichier : frontend/app/login/page.tsx — ligne 0

Correction : Réduire la fonction handleSubmit en utilisant des fonctions plus petites et plus faciles à comprendre, par exemple, une fonction pour gérer l'erreur et une autre pour gérer la connexion.

Maintenabilité

Sévérité : HAUTE

Fichier : frontend/app/profile/page.tsx — ligne 0

Correction : Réduire la fonction handleLogout en utilisant des fonctions plus petites et plus faciles à comprendre, par exemple, une fonction pour gérer la déconnexion et une autre pour gérer la redirection.

Maintenabilité

Sévérité : HAUTE

Fichier : frontend/app/register/page.tsx — ligne 0

Correction : Réduire la fonction handleSubmit en utilisant des fonctions plus petites et plus faciles à comprendre, par exemple, une fonction pour gérer l'erreur et une autre pour gérer l'inscription.

Maintenabilité

Sévérité : HAUTE

Fichier : frontend/app/reset-password/page.tsx — ligne 0

Correction : Réduire la fonction handleSubmit en utilisant des fonctions plus petites et plus faciles à comprendre, par exemple, une fonction pour gérer l'erreur et une autre pour gérer la réinitialisation.

Boucle $O(n^2)$

Sévérité : HAUTE

Fichier : frontend/app/explorer/page.tsx — ligne 1

Correction : Utiliser une structure de données plus efficace ou optimiser la boucle

Appels API externes à l'intérieur de boucles

Sévérité : HAUTE

Fichier : frontend/app/forgot-password/page.tsx — ligne 1

Correction : Utiliser un mécanisme de cache ou optimiser les appels API

Appels API externes à l'intérieur de boucles

Sévérité : HAUTE

Fichier : frontend/app/reset-password/page.tsx — ligne 1

Correction : Utiliser un mécanisme de cache ou optimiser les appels API

Duplication

Sévérité : MOYENNE

Fichier : backend/app/routes/analyses.py — ligne 0

Correction : Réduire la duplication de code en utilisant des fonctions ou des classes plus génériques.

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 15

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `get_db()`

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/app/routes/analyses.py — ligne 120

Correction : Ajouter un commentaire pour expliquer le but de la fonction `creer_issues_gitlab()`

Nommage ambigu

Sévérité : MOYENNE

Fichier : backend/app/routes/auth.py — ligne 50

Correction : Renommer la variable `'otp'` en `'otp_code'` pour clarifier son rôle

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 14

Correction : Utiliser une exception spécifique au lieu d'une exception générique (`Exception`, `Error`).

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/routes/analyses.py — ligne 143

Correction : Utiliser une exception spécifique au lieu d'une exception générique (`Exception`, `Error`).

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/routes/auth.py — ligne 123

Correction : Utiliser une exception spécifique au lieu d'une exception générique (Exception, Error).

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/routes/depots.py — ligne 134

Correction : Utiliser une exception spécifique au lieu d'une exception générique (Exception, Error).

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_client.py — ligne 24

Correction : Utiliser une exception spécifique au lieu d'une exception générique (Exception, Error).

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_service.py — ligne 6

Correction : Utiliser une exception spécifique au lieu d'une exception générique (Exception, Error).

Gestion erreurs

Sévérité : MOYENNE

Fichier : backend/app/services/llm_service.py — ligne 14

Correction : Utiliser une exception spécifique au lieu d'une exception générique (Exception, Error).

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/Admin_page/page.tsx — ligne 123

Correction : Utiliser des requêtes SQL avec des colonnes spécifiques au lieu de SELECT *

Appels API externes à l'intérieur de boucles

Sévérité : MOYENNE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 23

Correction : Utiliser des mécanismes de cache pour les données rarement modifiées

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/Admin_page/page.tsx — ligne 456

Correction : Ajouter un commentaire pour expliquer l'algorithme utilisé dans la fonction toggleActive

TODO non résolu

Sévérité : MOYENNE

Fichier : frontend/app/analyse/page.tsx — ligne 123

Correction : Résoudre le TODO 'Comparer les branches juste après création' dans la fonction handleSubmit

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/Admin_page/page.tsx — ligne 0

Correction : La fonction `loadAll` est trop longue et complexe. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à comprendre.

Gestion erreurs

Sévérité : MOYENNE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 0

Correction : Il est préférable de gérer les erreurs de manière plus spécifique et d'utiliser des exceptions personnalisées plutôt que des exceptions génériques.

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/analyse/page.tsx — ligne 0

Correction : La fonction `lancerAnalyse` est trop longue et complexe. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction `fetchDepots` en utilisant des fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction `handleLogout` en utilisant des fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction `handleDelete` en utilisant des fonctions plus petites et plus faciles à comprendre.

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 24

Correction : Utiliser des requêtes SQL avec des colonnes spécifiques au lieu de `SELECT *`

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 24

Correction : Utiliser des requêtes SQL avec des colonnes spécifiques au lieu de `SELECT *`

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 24

Correction : Utiliser des requêtes SQL avec des colonnes spécifiques au lieu de SELECT *

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 15

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `fetchDepots`

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 34

Correction : Ajouter un commentaire pour expliquer la logique derrière la fonction `buildTree`

TODO non résolu

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 15

Correction : Résoudre le TODO lié à la récupération des dépôts

TODO non résolu

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 30

Correction : Résoudre le TODO lié à la récupération des utilisateurs

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 15

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `fetchDepots`

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 34

Correction : Ajouter un commentaire pour expliquer la logique derrière la fonction `buildTree`

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 14

Correction : La fonction fetchDepots est trop longue et fait plusieurs choses. Il est préférable de la diviser en plusieurs fonctions plus petites et plus spécifiques.

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 34

Correction : La fonction fetchUser est trop longue et fait plusieurs choses. Il est préférable de la diviser en plusieurs fonctions plus petites et plus spécifiques.

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 24

Correction : La fonction fetchDepots est trop longue et fait plusieurs choses. Il est préférable de la diviser en plusieurs fonctions plus petites et plus spécifiques.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 140

Correction : La fonction fetchDepots utilise des variables globales qui rendent difficile la mise en place de tests unitaires.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 50

Correction : La fonction fetchUser utilise des variables globales qui rendent difficile la mise en place de tests unitaires.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 30

Correction : La fonction fetchDepots utilise des variables globales qui rendent difficile la mise en place de tests unitaires.

Architecture

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 150

Correction : La fonction fetchDepots utilise des requêtes HTTP qui pourraient être gérées de manière plus efficace en utilisant un service de gestion des requêtes.

Architecture

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 60

Correction : La fonction fetchUser utilise des requêtes HTTP qui pourraient être gérées de manière plus efficace en utilisant un service de gestion des requêtes.

Architecture

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 40

Correction : La fonction fetchDepots utilise des requêtes HTTP qui pourraient être gérées de manière plus efficace en utilisant un service de gestion des requêtes.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/explorer/page.tsx — ligne 0

Correction : Réduire la complexité de la fonction buildTree en utilisant des fonctions plus petites et plus faciles à comprendre.

SELECT*

Sévérité : MOYENNE

Fichier : frontend/app/difference/page.tsx — ligne 1

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Mémoire

Sévérité : MOYENNE

Fichier : frontend/app/explorer/page.tsx — ligne 1

Correction : Libérer les ressources mémoire inutiles

Calculs redondants

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 1

Correction : Mémoïser les calculs pour éviter les redondances

Calculs redondants

Sévérité : MOYENNE

Fichier : frontend/app/register/page.tsx — ligne 1

Correction : Mémoïser les calculs pour éviter les redondances

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/layout.tsx — ligne 1

Correction : Ajouter un commentaire pour expliquer le rôle de la fonction RootLayout

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/lib/api.ts — ligne 1

Correction : Ajouter un commentaire pour expliquer le rôle des fonctions loginUser et registerUser

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/page.tsx — ligne 1

Correction : Ajouter un commentaire pour expliquer le rôle de la fonction Home

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/explorer/page.tsx — ligne 44

Correction : La fonction `buildTree` est trop longue et complexe. Il est préférable de la diviser en plusieurs fonctions plus petites et plus faciles à comprendre.

Gestion erreurs

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 44

Correction : Il est préférable de gérer les erreurs de manière plus spécifique, plutôt que de simplement afficher un message d'erreur générique.

Calculs redondants

Sévérité : FAIBLE

Fichier : frontend/app/analyse/page.tsx — ligne 123

Correction : Mémoïser les calculs pour éviter les recalculs inutiles

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/Admin_page/page.tsx — ligne 123

Correction : Ajouter une docstring pour expliquer le rôle de la fonction DonutChart

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 23

Correction : Renommer la variable 'token' en 'gitlabToken' pour clarifier son rôle

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/components/Button.tsx — ligne 0

Correction : Utiliser des noms de variables plus significatifs, comme `buttonType` au lieu de `type`.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/components/Input.tsx — ligne 0

Correction : Utiliser des noms de variables plus significatifs, comme ``inputType`` au lieu de ``type``.

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/components/Button.tsx — ligne 5

Correction : Considérer de renommer la fonction ``Button`` en ``IconButton`` pour refléter son rôle

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/components/Input.tsx — ligne 5

Correction : Considérer de renommer la fonction ``Input`` en ``TextInput`` pour refléter son rôle

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/explorer/page.tsx — ligne 0

Correction : Utiliser des variables plus significatives pour les variables locales, par exemple, au lieu de `'i'`, utiliser `'index'` ou `'iteration'`.

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/difference/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `DifferencePage`

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/explorer/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `ExplorerPage`

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/forgot-password/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction ForgotPassword

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/login/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction LoginPage

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/profile/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction ProfilePage

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/register/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction RegisterPage

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le rôle de la fonction ResetPassword

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/register/page.tsx — ligne 1

Correction : Renommer la variable confirmPassword en confirm_password pour clarifier son rôle

Gestion erreurs

Sévérité : FAIBLE

Fichier : frontend/app/profile/page.tsx — ligne 134

Correction : Il est préférable de gérer les erreurs de manière plus spécifique, plutôt que de simplement afficher un message d'erreur générique.

Architecture

Sévérité : FAIBLE

Fichier : frontend/app/layout.tsx — ligne 10

Correction : Il est préférable de séparer la logique de présentation de la logique métier, pour améliorer la maintenabilité et la testabilité du code.

Architecture

Sévérité : FAIBLE

Fichier : frontend/app/lib/api.ts — ligne 10

Correction : Il est préférable de séparer la logique de présentation de la logique métier, pour améliorer la maintenabilité et la testabilité du code.

■ Recommandations

1. Améliorer la lisibilité du code

Utiliser des noms de variables et de fonctions plus explicites et éviter les longues lignes de code.

2. Améliorer la maintenabilité du code

Utiliser des principes de conception SOLID et éviter les couplages forts entre les modules.

3. Utiliser un gestionnaire de contexte pour fermer les connexions HTTP

Utilisez un gestionnaire de contexte pour fermer les connexions HTTP et éviter les problèmes de performance

4. Ajouter des commentaires explicatifs

Ajouter des commentaires pour expliquer les raisons derrière les choix de conception et les algorithmes utilisés dans le code.

5. Utiliser des noms de variables clairs

Utiliser des noms de variables qui reflètent leur contenu et leur rôle dans le code.

6. Utiliser des exceptions spécifiques

Utiliser des exceptions spécifiques au lieu d'exceptions génériques (Exception, Error) pour améliorer la gestion des erreurs et faciliter le débogage.

7. Utiliser des variables d'environnement

Utiliser des variables d'environnement au lieu de valeurs codées en dur pour améliorer la flexibilité et la sécurité de l'application.

8. Utiliser des requêtes SQL avec des paramètres

Utiliser des requêtes SQL avec des paramètres pour éviter les injections SQL et améliorer la sécurité

9. Utiliser des mécanismes de cache

Utiliser des mécanismes de cache pour les données rarement modifiées pour améliorer les performances

10. Mémoiser les calculs

Mémoiser les calculs pour éviter les recalculs inutiles et améliorer les performances

11. Résoudre les TODO non résolus

Résoudre les TODO non résolus pour améliorer la qualité et la fiabilité du code

12. Décomposer les fonctions et classes trop grandes

Il est préférable de décomposer les fonctions et classes trop grandes en plusieurs plus petites et plus faciles à comprendre. Cela améliorera la lisibilité et la maintenabilité du code.

13. Gérer les erreurs de manière plus spécifique

Il est préférable de gérer les erreurs de manière plus spécifique et d'utiliser des exceptions personnalisées plutôt que des exceptions génériques. Cela améliorera la lisibilité et la maintenabilité du code.

14. Utiliser des fonctions plus petites et plus faciles à comprendre

La complexité de certaines fonctions est trop élevée, ce qui rend difficile leur compréhension et leur maintenance. Il est recommandé de les réduire en utilisant des fonctions plus petites et plus faciles à comprendre.

15. Utiliser des noms de variables plus significatifs

Les noms de variables utilisés dans certaines fonctions sont trop courts ou trop vagues, ce qui rend difficile leur compréhension. Il est recommandé d'utiliser des noms de variables plus significatifs et plus explicites.

16. Optimisation des requêtes SQL

Utilisez des requêtes SQL avec des colonnes spécifiques au lieu de `SELECT *` pour réduire la charge sur la base de données. Utilisez également des jointures pour réduire le nombre de requêtes SQL.

17. Utilisation de l'indexation

Assurez-vous que les colonnes utilisées dans les `WHERE`, `JOIN` et `ORDER BY` sont indexées pour améliorer les performances des requêtes SQL.

18. Gestion des connexions

Fermez les connexions à la base de données après utilisation pour éviter les fuites de mémoire.

19. Améliorer la nomenclature

Améliorer la nomenclature pour refléter les rôles des fonctions et des variables

20. Diviser les fonctions en plusieurs plus petites et plus spécifiques

Cela rendra le code plus facile à comprendre et à maintenir. Il est préférable de diviser les fonctions en plusieurs plus petites et plus spécifiques pour éviter les fonctions trop longues et complexes.

21. Utiliser des variables locales et non globales

Cela rendra le code plus facile à comprendre et à maintenir. Il est préférable d'utiliser des variables locales et non globales pour éviter les variables globales qui rendent difficile la mise en place de tests unitaires.

22. Utiliser des services de gestion des requêtes

Cela rendra le code plus efficace et plus facile à maintenir. Il est préférable d'utiliser des services de gestion des requêtes pour gérer les requêtes HTTP de manière plus efficace.

23. Utiliser des variables plus significatives

Cela améliorera la lisibilité et la maintenabilité du code, en permettant aux développeurs de comprendre plus facilement le fonctionnement du code et de le modifier plus facilement.

24. Utiliser les colonnes nécessaires

Lorsque vous effectuez des requêtes SQL, utilisez les colonnes nécessaires au lieu de `SELECT *` pour réduire la quantité de données transférées et améliorer les performances.

25. Optimiser les boucles

Les boucles imbriquées peuvent entraîner des complexités $O(n^2)$ ou pires. Utilisez des structures de données plus efficaces ou optimisez les boucles pour améliorer les performances.

26. Libérer les ressources mémoire

Libérez les ressources mémoire inutiles pour éviter les fuites mémoire et améliorer les performances.

27. Utiliser un mécanisme de cache

Utilisez un mécanisme de cache pour stocker les résultats des appels API externes et réduire le nombre de requêtes réseau.

28. Ajouter des docstrings pour les fonctions

Ajouter des docstrings pour expliquer le rôle de chaque fonction, notamment pour les fonctions `DifferencePage`, `ExplorerPage`, `ForgotPassword`, `LoginPage`, `ProfilePage`, `RegisterPage` et `ResetPassword`

29. Ajouter des commentaires pour les fonctions

Ajouter des commentaires pour expliquer le rôle de chaque fonction, notamment pour les fonctions `RootLayout`, `loginUser`, `registerUser`, `Home` et les fonctions de la page de registration

30. Renommer les variables pour clarifier leur rôle

Renommer les variables pour clarifier leur rôle, notamment pour la variable `confirmPassword` dans la page de registration

31. Séparer la logique de présentation de la logique métier

Il est préférable de séparer la logique de présentation de la logique métier, pour améliorer la maintenabilité et la testabilité du code. Cela peut être réalisé en créant des composants de présentation séparés des composants métier.